

OPS-SIG

Algorithm Specifications and Supporting Documentation
(version 1.0)

Yunlei Zhao, Yixuan Song, Yifan Dong, Zhaohui Cheng,
Jieyu Zheng, Han Liu, Shuaihao Li, Yu Wu, Wenbo Guo

June 29, 2026

Contents

1	Algorithm Overview	2
1.1	Scheme framework	2
1.2	Main design parameters	2
2	Notation and Definitions	2
2.1	Rings, reductions, and norms	2
2.2	Sets and distributions	3
2.3	Hash and extendable-output functions	3
2.4	Message and context encoding	3
2.5	Keys, signatures, and correctness	3
3	Algorithm Description	4
3.1	OPS Signature Scheme	4
3.1.1	Key generation	4
3.1.2	Signing	5
3.1.3	Verification	6
3.1.4	Correctness	6
3.2	Auxiliary Functions	7
3.2.1	Conversion between data types	7
3.2.2	Encodings of keys and signatures	8
3.2.3	Pseudorandom sampling	12
3.2.4	High-order and low-order bits and hints	13
3.2.5	NTT and inverse NTT	14
3.2.6	Parallel small polynomial multiplications	16
3.2.7	Arithmetic under NTT	18
4	Design Rationale	18
4.1	Underlying hardness assumptions	18
4.2	Parameter selection	19
4.3	Failure probability	19
5	Security Statement and Analysis	19
5.1	Security claims	19
5.2	Security model	19
5.3	Hardness assumptions	19
5.4	Theoretical security in the ROM	20
5.5	Theoretical security in the QROM	23
5.6	Concrete security	24
6	Performance Evaluation	25
6.1	Asymptotic performance	25
6.2	Software benchmark results (reference implementation)	26
7	Feature Statement	26
7.1	Declared features	26
7.2	Security declarations	27

1 Algorithm Overview

1.1 Scheme framework

OPS-SIG is a symmetric module-lattice-based digital signature scheme instantiated from the MLWE and MSIS problems. Its design follows the well-studied “Fiat–Shamir with abort” structure of ML-DSA (Dilithium-style) signatures [1] while adopting a module setting to reduce sizes [2].

At a high level, the scheme proceeds in three main steps:

- **Key generation (OPS-SIG.KeyGen)** samples short secrets, expands a public random module matrix A , and computes a rounded public value.
- **Signing (OPS-SIG.Sign)** uses rejection sampling to produce a response independent of secret information beyond what is proven secure, and outputs a signature $(\tilde{c}, \mathbf{z}, \mathbf{h})$.
- **Verification (OPS-SIG.Verify)** recomputes a commitment from $(\tilde{c}, \mathbf{z}, \mathbf{h})$ and checks consistency via hashing.

The reference implementation in this repository currently realizes the Level-1 instance (128-bit classical security target). This specification is parameterized and covers three instances; where packing and sampling depend on $(k, \ell, n, \eta, \gamma_1, \gamma_2)$, we define them generically.

1.2 Main design parameters

Table 1 lists the three supported instances. Here n is the ring dimension, $q = 2^{26} - 2^{12} + 1 = 67104769$ is the modulus, (k, ℓ) are module ranks, (η_1, η_2) are secret-noise bounds (in this submission we use $\eta_1 = \eta_2 = \eta$), γ_1, γ_2 are decomposition parameters, d is the rounding base parameter, τ is the Hamming weight of the challenge polynomial, and ω bounds the number of hint bits in the signature. The column “Rep.” is the empirical expected number of sampling iterations in the signing rejection loop.

Table 1: Algorithm instances and parameters.

Instance	n	q	(k, ℓ)	(η_1, η_2)	γ_1	γ_2	d	τ	ω	Rep.	$(\mathbf{pk} , \mathbf{sk} , \mathbf{sig})$
Level-1 (128/80)	512	67104769	(3,3)	(2,2)	2^{21}	$(q-1)/32$	13	39	80	1.12	(2560, 3840, 4349)
Level-3 (256/128)	512	67104769	(4,4)	(3,3)	2^{20}	$(q-1)/96$	13	45	85	1.93	(3392, 5056, 5540)
Level-5 (512/256)	1024	67104769	(4,4)	(2,2)	2^{22}	$(q-1)/48$	13	90	90	2.02	(6720, 9920, 12021)

2 Notation and Definitions

2.1 Rings, reductions, and norms

Let n be a power of two and let q be an odd integer. Define the rings

$$\mathcal{R} = \mathbb{Z}[X]/(X^n + 1), \quad \mathcal{R}_q = \mathbb{Z}_q[X]/(X^n + 1).$$

For an integer a , define the non-negative remainder $a \bmod^+ q \in \{0, 1, \dots, q-1\}$ and the centered remainder $a \bmod^\pm q \in \left\{-\frac{q-1}{2}, \dots, \frac{q-1}{2}\right\}$. For a polynomial $w = \sum_{i=0}^{n-1} w_i X^i \in \mathcal{R}_q$, we interpret each coefficient using the centered representative $w_i \bmod^\pm q$ when measuring sizes.

For $w \in \mathcal{R}_q$, define

$$\|w\|_\infty := \max_{0 \leq i < n} |w_i \bmod^\pm q|.$$

For a vector $\mathbf{w} = (w_0, \dots, w_{\ell-1}) \in \mathcal{R}_q^\ell$ (or $\mathcal{R}_q^k$), define $\|\mathbf{w}\|_\infty := \max_i \|w_i\|_\infty$.

2.2 Sets and distributions

For an integer $\eta \geq 1$, let

$$S_\eta := \{a \in \mathbb{Z} : |a| \leq \eta\}, \quad S_\eta^\ell := \{\mathbf{s} \in \mathcal{R}^\ell : \|\mathbf{s}\|_\infty \leq \eta\}.$$

For γ_1 , define

$$\tilde{S}_{\gamma_1} := \{a \in \mathbb{Z} : 1 - \gamma_1 \leq a \leq \gamma_1\}, \quad \tilde{S}_{\gamma_1}^\ell := \{\mathbf{y} \in \mathcal{R}^\ell : \|\mathbf{y}\|_\infty \leq \gamma_1\}.$$

Let $\mathcal{B}_\tau \subset \mathcal{R}$ denote the set of polynomials with exactly τ coefficients in $\{+1, -1\}$ and all other coefficients equal to 0.

2.3 Hash and extendable-output functions

The scheme uses a hash function H and an extendable-output function (XOF) denoted **Sample**. The reference code instantiates these with SM3-based constructions (see [3, 4]) for correctness testing; a production instantiation must use cryptographically secure primitives.

We use the following interfaces:

- $H_\lambda(x)$ returns λ bytes.
- $\text{Sample}(x, \text{outlen})$ returns outlen bytes.
- $\text{CRH}(x) := H_{128}(x)$ is a 1024-bit hash used for commitment hashing.

2.4 Message and context encoding

The signing and verification algorithms accept a context string ctx of length at most 255 bytes. Define

$$\text{pre} := 0x00 \parallel \text{len}(\text{ctx}) \parallel \text{ctx}.$$

Messages may be up to 2^{63} bits, and a keypair supports at least 2^{64} distinct signing and verification queries.

2.5 Keys, signatures, and correctness

Let $\rho \in \{0, 1\}^{512}$ be the seed defining the public matrix $A \in \mathcal{R}_q^{k \times \ell}$, and let $\text{tr} := \text{CRH}(\text{pk})$. The public key is $\text{pk} = (\rho, t_1)$ and the secret key is $\text{sk} = (\rho, \text{tr}, \mathbf{s}_1, \mathbf{s}_2, \mathbf{t}_0)$. A signature is $\sigma = (\tilde{c}, \mathbf{z}, \mathbf{h})$ where $\tilde{c} \in \{0, 1\}^{2\lambda}$ is a challenge seed, $\mathbf{z} \in \mathcal{R}^\ell$ is the response, and $\mathbf{h} \in \{0, 1\}^{k \times n}$ is a sparse hint vector with at most ω nonzero entries.

Concrete byte encodings are defined by **Pack** and **Unpack** algorithms (Section 3.2.2) and depend on $(k, \ell, n, \eta, \gamma_1, \gamma_2, \omega)$.

3 Algorithm Description

3.1 OPS Signature Scheme

3.1.1 Key generation

Algorithm 1 OPS-SIG.KeyGen(1^λ)

Output: public key $\mathbf{pk}$ and secret key $\mathbf{sk}$

- 1: $(\rho, \rho') \leftarrow \{0, 1\}^{512} \times \{0, 1\}^{1024}$
 - 2: $A \leftarrow \text{ExpandA}(\rho) \in \mathcal{R}_q^{k \times \ell}$
 - 3: $\mathbf{s}_1 \leftarrow S_{\eta_1}^\ell$ using ρ'
 - 4: $\mathbf{s}_2 \leftarrow S_{\eta_2}^k$ using ρ'
 - 5: $\mathbf{t} \leftarrow A\mathbf{s}_1 + \mathbf{s}_2 \in \mathcal{R}_q^k$
 - 6: $(\mathbf{t}_1, \mathbf{t}_0) \leftarrow \text{Power2Round}(\mathbf{t})$
 - 7: $\mathbf{pk} \leftarrow \text{PackPK}(\rho, \mathbf{t}_1)$
 - 8: $\text{tr} \leftarrow \text{CRH}(\mathbf{pk})$
 - 9: $\mathbf{sk} \leftarrow \text{PackSK}(\rho, \text{tr}, \mathbf{s}_1, \mathbf{s}_2, \mathbf{t}_0)$
 - 10: **return** $(\mathbf{pk}, \mathbf{sk})$
-

Algorithm 1 samples a 512-bit seed ρ and a 1024-bit seed ρ' . The seed ρ deterministically defines a uniformly random public matrix $A \in \mathcal{R}_q^{k \times \ell}$ via ExpandA . Using the seed ρ' , the algorithm samples short secret vectors $\mathbf{s}_1 \in S_{\eta_1}^\ell$ and $\mathbf{s}_2 \in S_{\eta_2}^k$ and computes $\mathbf{t} = A\mathbf{s}_1 + \mathbf{s}_2 \in \mathcal{R}_q^k$. The rounding procedure Power2Round splits $\mathbf{t}$ into a high part $\mathbf{t}_1$ and a low part $\mathbf{t}_0$ with base 2^d . The public key is the encoding of $(\rho, \mathbf{t}_1)$; the secret key stores $(\rho, \text{tr}, \mathbf{s}_1, \mathbf{s}_2, \mathbf{t}_0)$, where $\text{tr} := \text{CRH}(\mathbf{pk})$ binds the secret key to its public key.

3.1.2 Signing

Algorithm 2 OPS-SIG.Sign(sk, M, ctx)

Input: secret key sk, message $M \in \{0, 1\}^*$, context string ctx

Output: signature σ

```

1: Parse sk as  $(\rho, \text{tr}, \mathbf{s}_1, \mathbf{s}_2, \mathbf{t}_0) \leftarrow \text{UnpackSK}(\text{sk})$ 
2:  $\mathbf{T}_{\mathbf{s}_1} \leftarrow \text{PrepareTable}(\mathbf{s}_1, \eta_1, \ell)$ 
3:  $\mathbf{T}_{\mathbf{s}_2} \leftarrow \text{PrepareTable}(\mathbf{s}_2, \eta_2, k)$ 
4:  $\text{pre} \leftarrow 0x00 \parallel \text{len}(\text{ctx}) \parallel \text{ctx}$ 
5:  $\mu \leftarrow \text{CRH}(\text{tr} \parallel \text{pre} \parallel M)$ 
6:  $\rho'' \leftarrow \{0, 1\}^{1024}$ 
7:  $A \leftarrow \text{ExpandA}(\rho) \in \mathcal{R}_q^{k \times \ell}$ 
8:  $\kappa \leftarrow 0$ 
9: loop
10:  $\mathbf{y} \leftarrow \tilde{S}_{\gamma_1}^\ell$  using  $(\rho'', \kappa)$ ;  $\kappa \leftarrow \kappa + 1$ 
11:  $\mathbf{w} \leftarrow A\mathbf{y} \in \mathcal{R}_q^k$ 
12:  $(\mathbf{w}_1, \mathbf{w}_0) \leftarrow \text{HighBits/LowBits}(\mathbf{w})$ 
13:  $\tilde{c} \leftarrow H_{2\lambda}(\mu \parallel \text{W1Encode}(\mathbf{w}_1))$ 
14:  $c \leftarrow \text{SampleInBall}(\tilde{c}) \in \mathcal{B}_\tau$ 
15: if SmallPolywithEarlycheck( $\mathbf{z}, \mathbf{w}'_0, c, \mathbf{T}_{\mathbf{s}_1}, \mathbf{T}_{\mathbf{s}_2}, \mathbf{w}_0, \mathbf{y}, \gamma_1 - \beta_1, \gamma_2 - \beta_2$ ) then
16:   continue
17: end if
18:  $\mathbf{h}' \leftarrow \text{SmallPoly}(c, \mathbf{t}_0, 2^{d-1}, k)$ 
19: if  $\|\mathbf{h}'\|_\infty \geq \gamma_2$  then
20:   continue
21: end if
22:  $\mathbf{w}''_0 \leftarrow \mathbf{w}'_0 + \mathbf{h}'$ 
23:  $\mathbf{h} \leftarrow \text{MakeHint}(\mathbf{w}''_0, \mathbf{w}_1)$ 
24: if  $\|\mathbf{h}\|_1 > \omega$  then
25:   continue
26: end if
27:  $\sigma \leftarrow \text{PackSig}(\tilde{c}, \mathbf{z}, \mathbf{h})$ 
28: return  $\sigma$ 
29: end loop

```

The signing algorithm OPS-SIG.Sign is shown in Algorithm 2. It takes as input a secret key sk and a message M (with an optional context string ctx), and outputs a signature $\sigma = (\tilde{c}, \mathbf{z}, \mathbf{h})$. First, it parses sk to recover the matrix seed ρ , the public key hash tr, the secret vectors $\mathbf{s}_1, \mathbf{s}_2$, and $\mathbf{t}_0$. Immediately afterwards, the signer computes the precomputed tables $\mathbf{T}_{\mathbf{s}_1}, \mathbf{T}_{\mathbf{s}_2}$ for the secret vectors $\mathbf{s}_1$ and $\mathbf{s}_2$. It then computes $\mu \in \{0, 1\}^{1024}$ as $\mu := \text{CRH}(\text{tr} \parallel \text{pre} \parallel M)$, where pre encodes the context.

Next, it samples a randomness seed $\rho'' \leftarrow \{0, 1\}^{1024}$. The seed ρ'' drives the rejection-sampling loop. For each iteration counter κ , the signer samples a fresh ephemeral vector $\mathbf{y} \in \tilde{S}_{\gamma_1}^\ell$ and computes $\mathbf{w} := A\mathbf{y} \in \mathcal{R}_q^k$ using the public matrix $A = \text{ExpandA}(\rho)$. The vector $\mathbf{w}$ is decomposed into high bits $\mathbf{w}_1$ and low bits $\mathbf{w}_0$ with respect to $2\gamma_2$, written $(\mathbf{w}_1, \mathbf{w}_0) := (\text{HighBits}_q(\mathbf{w}, 2\gamma_2), \text{LowBits}_q(\mathbf{w}, 2\gamma_2))$. The signer then computes a challenge seed $\tilde{c} := H_{2\lambda}(\mu \parallel \text{W1Encode}(\mathbf{w}_1))$ and samples a sparse challenge polynomial $c \in \mathcal{B}_\tau$ using SampleInBall, where λ denotes the quantum security parameter in bytes.

The response is computed conditionally. To ensure security and correctness, the signer computes SmallPolywithEarlycheck($\mathbf{z}, \mathbf{w}'_0, c, \mathbf{T}_{\mathbf{s}_1}, \mathbf{T}_{\mathbf{s}_2}, \mathbf{w}_0, \mathbf{y}, \gamma_1 - \beta_1, \gamma_2 - \beta_2$); if it fails, the algorithm

restarts with the next κ . It also computes $\mathbf{h}' := \text{SmallPoly}(c, \mathbf{t}_0, 2^d, k)$ and checks $\|\mathbf{h}'\|_\infty < \gamma_2$. If all checks pass, it computes the hint vector $\mathbf{h} := \text{MakeHint}_q(\mathbf{w}_0'', \mathbf{w}_1)$ where $\mathbf{w}_0'' := \mathbf{w}_0' + \mathbf{h}'$, and enforces that $\mathbf{h}$ has at most ω nonzero coefficients (i.e., $\|\mathbf{h}\|_1 \leq \omega$). Finally, it outputs $\sigma = \text{PackSig}(\tilde{c}, \mathbf{z}, \mathbf{h})$.

Implementations may combine the computations of $\mathbf{z}$, $\mathbf{w}_0'$, and the associated norm checks to reject early (an “early-check” optimization) without changing the mathematical specification. Such optimizations only affect performance; they do not change the output distribution conditioned on acceptance.

3.1.3 Verification

Algorithm 3 OPS-SIG.Verify(pk, M, ctx, σ)

Input: public key pk, message $M \in \{0, 1\}^*$, context string ctx, signature σ

Output: accept or reject

```

1: Parse pk as  $(\rho, \mathbf{t}_1) \leftarrow \text{UnpackPK}(\text{pk})$ 
2: Parse  $\sigma$  as  $(\tilde{c}, \mathbf{z}, \mathbf{h}) \leftarrow \text{UnpackSig}(\sigma)$ 
3: if  $\|\mathbf{z}\|_\infty \geq \gamma_1 - \beta_1$  then
4:   return reject
5: end if
6:  $\text{pre} \leftarrow 0x00 \parallel \text{len}(\text{ctx}) \parallel \text{ctx}$ 
7:  $\text{tr} \leftarrow \text{CRH}(\text{pk})$ 
8:  $\mu \leftarrow \text{CRH}(\text{tr} \parallel \text{pre} \parallel M)$ 
9:  $A \leftarrow \text{ExpandA}(\rho) \in \mathcal{R}_q^{k \times \ell}$ 
10:  $c \leftarrow \text{SampleInBall}(\tilde{c})$ 
11:  $\mathbf{w} \leftarrow A\mathbf{z} - c\mathbf{t}_1 \cdot 2^d$ 
12:  $\mathbf{w}_1 \leftarrow \text{UseHint}(\mathbf{w}, \mathbf{h})$ 
13:  $\tilde{c}' \leftarrow H_{2\lambda}(\mu \parallel \text{W1Encode}(\mathbf{w}_1))$ 
14: if  $\tilde{c}' = \tilde{c}$  then
15:   return accept
16: else
17:   return reject
18: end if

```

The verification algorithm OPS-SIG.Verify is shown in Algorithm 3. It parses the public key to obtain the matrix seed ρ and the rounded vector $\mathbf{t}_1$. It parses the signature $(\tilde{c}, \mathbf{z}, \mathbf{h})$ and rejects immediately if $\|\mathbf{z}\|_\infty \geq \gamma_1 - \beta_1$. It then recomputes $\mu := \text{CRH}(\text{CRH}(\text{pk}) \parallel \text{pre} \parallel M)$ and expands the public matrix $A = \text{ExpandA}(\rho)$. From $\tilde{c}$ it reconstructs the challenge polynomial $c \in \mathcal{B}_\tau$.

Using the linear relation underlying the scheme, the verifier reconstructs

$$\mathbf{w} := A\mathbf{z} - c\mathbf{t}_1 \cdot 2^d.$$

It then uses the hint vector $\mathbf{h}$ to recover the high bits $\mathbf{w}_1$ via UseHint , and recomputes the challenge seed $\tilde{c}' := H_{2\lambda}(\mu \parallel \text{W1Encode}(\mathbf{w}_1))$. The signature is accepted if and only if $\tilde{c}' = \tilde{c}$.

3.1.4 Correctness

Theorem 1 (Correctness). *For any security parameter and any message M and context ctx, if $(pk, sk) \leftarrow \text{OPS-SIG.KeyGen}(1^\lambda)$ and $\sigma \leftarrow \text{OPS-SIG.Sign}(sk, M, \text{ctx})$, then*

$$\text{OPS-SIG.Verify}(pk, M, \text{ctx}, \sigma) = \text{accept}.$$

Proof sketch. Write $\mathbf{t} = A\mathbf{s}_1 + \mathbf{s}_2$ and $(\mathbf{t}_1, \mathbf{t}_0) = \text{Power2Round}(\mathbf{t})$, so that $\mathbf{t} = \mathbf{t}_1 \cdot 2^d + \mathbf{t}_0$. In signing, $\mathbf{z} = \mathbf{y} + c\mathbf{s}_1$ and $\mathbf{w} = A\mathbf{y}$. Therefore

$$A\mathbf{z} - c(\mathbf{t}_1 \cdot 2^d) = A(\mathbf{y} + c\mathbf{s}_1) - c(\mathbf{t} - \mathbf{t}_0) = \mathbf{w} - c\mathbf{s}_2 + c\mathbf{t}_0.$$

The decomposition and hint procedures ensure that the verifier reconstructs the same high bits $\mathbf{w}_1$ used by the signer (when the rejection tests pass), so the recomputed challenge seed matches, and verification accepts. $\square$

3.2 Auxiliary Functions

This section specifies the auxiliary functions used by OPS-SIG. The presentation follows the structure of the ML-DSA auxiliary-function specification [1] while adapting definitions to the parameters and ring $\mathcal{R}_q$ used here.

3.2.1 Conversion between data types

Algorithm 4 BytesToBits(B)

Input: byte string B of length ℓ_B

Output: bit string $b_0 \dots b_{8\ell_B-1}$ in little-endian bit order per byte

1: Interpret B as a byte string of length ℓ_B

2: **return** the bit string $b_0 \dots b_{8\ell_B-1}$ in little-endian bit order per byte

Algorithm 5 BitsToBytes($b_0 \dots b_{8\ell_B-1}$)

Input: bit string $b_0 \dots b_{8\ell_B-1}$, where $8\ell_B$ is a multiple of 8

Output: byte string B of length ℓ_B

1: Pack bits into ℓ_B bytes using little-endian bit order per byte

2: **return** the resulting byte string

The conversion routines define the canonical mapping between byte strings and bit strings. All encodings in this document specify exact bit lengths; **BitsToBytes** is applied only when the total bit length is a multiple of 8.

Algorithm 6 SampleInBall($\tilde{c}$)

Input: seed $\tilde{c} \in \{0, 1\}^{2\lambda}$
Output: $c \in \mathcal{B}_\tau$

- 1: $buf \leftarrow \text{Sample}(\tilde{c}, \text{outlen})$
- 2: Initialize $c \leftarrow 0$ and an array $idx[0..n-1] \leftarrow (0, 1, \dots, n-1)$
- 3: Parse τ sign bits from buf to obtain signs $s_0, \dots, s_{\tau-1} \in \{+1, -1\}$
- 4: **for** $t = 0$ to $\tau - 1$ **do**
- 5: $i \leftarrow n - \tau + t$
- 6: **loop**
- 7: Parse a 16-bit little-endian word x from buf ; if exhausted, extend buf with more
Sample output
- 8: $m \leftarrow x \cdot (i + 1)$ as a 32-bit integer
- 9: $j \leftarrow \lfloor m/2^{16} \rfloor$
- 10: $lo \leftarrow m \bmod 2^{16}$
- 11: $thr \leftarrow 2^{16} \bmod (i + 1)$
- 12: **if** $lo \geq thr$ **then**
- 13: **break**
- 14: **end if**
- 15: **end loop**
- 16: Swap $idx[i]$ and $idx[j]$
- 17: Set coefficient $c_{idx[j]} \leftarrow s_t$
- 18: **end for**
- 19: **return** $c \in \mathcal{B}_\tau$

This SampleInBall procedure deterministically maps $\tilde{c}$ to a challenge polynomial of Hamming weight τ . It selects τ distinct positions using a partial Fisher–Yates shuffle and assigns independent ± 1 signs. The bounded-index step uses a multiplication-based range reduction that avoids division and substantially reduces rejection probability for the supported dimensions $n \in \{512, 1024\}$.

3.2.2 Encodings of keys and signatures

Algorithm 7 PackPK($\rho, \mathbf{t}_1$)

Input: $\rho \in \{0, 1\}^{512}$, $\mathbf{t}_1 \in \mathcal{R}_q^k$
Output: public key byte string $\mathbf{pk}$

- 1: Encode ρ as 64 bytes
- 2: Encode each polynomial in $\mathbf{t}_1 \in \mathcal{R}_q^k$ using Poly1Pack (parameterized by n, q, d)
- 3: **return** concatenation

Algorithm 8 UnpackPK($\mathbf{pk}$)

Input: public key byte string $\mathbf{pk}$
Output: $(\rho, \mathbf{t}_1)$ with $\rho \in \{0, 1\}^{512}$, $\mathbf{t}_1 \in \mathcal{R}_q^k$

- 1: Parse first 64 bytes as ρ
- 2: Decode the remainder into $\mathbf{t}_1$ using Poly1Unpack
- 3: **return** $(\rho, \mathbf{t}_1)$

Algorithm 9 PackSK($\rho, \text{tr}, \mathbf{s}_1, \mathbf{s}_2, \mathbf{t}_0$)

Input: $\rho \in \{0, 1\}^{512}$, $\text{tr} \in \{0, 1\}^{1024}$, $\mathbf{s}_1 \in \mathcal{R}^\ell$, $\mathbf{s}_2 \in \mathcal{R}^k$, $\mathbf{t}_0 \in \mathcal{R}^k$

Output: secret key byte string sk

- 1: Encode ρ as 64 bytes each; encode tr as 128 bytes
 - 2: Encode each polynomial in $\mathbf{s}_1 \in \mathcal{R}^\ell$ and $\mathbf{s}_2 \in \mathcal{R}^k$ using PolyEtaPack (depends on η)
 - 3: Encode each polynomial in $\mathbf{t}_0 \in \mathcal{R}^k$ using Polyt0Pack (depends on d)
 - 4: **return** concatenation
-

Algorithm 10 UnpackSK(sk)

Input: secret key byte string sk

Output: $(\rho, \text{key}, \text{tr}, \mathbf{s}_1, \mathbf{s}_2, \mathbf{t}_0)$

- 1: Parse and decode in the reverse order of PackSK
 - 2: **return** $(\rho, \text{key}, \text{tr}, \mathbf{s}_1, \mathbf{s}_2, \mathbf{t}_0)$
-

Algorithm 11 PackSig($\tilde{c}, \mathbf{z}, \mathbf{h}$)

Input: $\tilde{c} \in \{0, 1\}^{2\lambda}$, $\mathbf{z} \in \mathcal{R}^\ell$, $\mathbf{h} \in \{0, 1\}^{k \times n}$

Output: signature byte string σ

- 1: Encode $\tilde{c}$ as $\lambda/4$ bytes
 - 2: Encode each polynomial in $\mathbf{z} \in \mathcal{R}^\ell$ using PolyzPack (depends on γ_1)
 - 3: Encode sparse $\mathbf{h}$ using HintPack (depends on ω)
 - 4: **return** concatenation
-

Algorithm 12 UnpackSig(σ)

Input: signature byte string σ

Output: $(\tilde{c}, \mathbf{z}, \mathbf{h})$ or reject

- 1: Parse $\tilde{c}$ and decode $\mathbf{z}$; decode $\mathbf{h}$ and reject if invalid
 - 2: **return** $(\tilde{c}, \mathbf{z}, \mathbf{h})$
-

The encodings above define the byte-level formats of public keys, secret keys, and signatures. The reference implementation provides concrete bit layouts for each packed polynomial type; these layouts differ across parameter sets because $(n, k, \ell, \eta, \gamma_1, \gamma_2, \omega)$ affect coefficient ranges and sparsity.

Algorithm 13 PolyEtaPack(a)

Input: polynomial $a \in \mathcal{R}$ with coefficients in $[-\eta, \eta]$

Output: byte string encoding of a

- 1: For each coefficient $a_i \in [-\eta, \eta]$, set $t_i \leftarrow \eta - a_i \in \{0, \dots, 2\eta\}$
 - 2: Let $b \leftarrow \lceil \log_2(2\eta + 1) \rceil$ (for $\eta \in \{2, 3\}$, $b = 3$)
 - 3: Pack t_i using b bits per coefficient in little-endian bit order
 - 4: **return** the packed byte string
-

Algorithm 14 PolyEtaUnpack(B)

Input: byte string B

Output: polynomial $a \in \mathcal{R}$ with coefficients in $[-\eta, \eta]$

- 1: Let $b \leftarrow \lceil \log_2(2\eta + 1) \rceil$ (for $\eta \in \{2, 3\}$, $b = 3$)
 - 2: Unpack b -bit integers t_i and set $a_i \leftarrow \eta - t_i$
 - 3: **return** a
-

Algorithm 15 Polyt1Pack(a)

Input: polynomial $a \in \mathcal{R}_q$ with coefficients in $\{0, \dots, 2^{\lceil \log_2(q) \rceil - d} - 1\}$

Output: byte string encoding of a

- 1: Let $b \leftarrow \lceil \log_2(q) \rceil - d$ (for $q = 67104769$ and $d = 13$, $b = 13$)
 - 2: Pack each coefficient $a_i \in \{0, \dots, 2^b - 1\}$ using b bits
 - 3: **return** the packed byte string
-

Algorithm 16 Polyt1Unpack(B)

Input: byte string B

Output: polynomial $a \in \mathcal{R}_q$ with coefficients in $\{0, \dots, 2^{\lceil \log_2(q) \rceil - d} - 1\}$

- 1: Let $b \leftarrow \lceil \log_2(q) \rceil - d$
 - 2: Unpack b -bit integers to obtain coefficients of a
 - 3: **return** a
-

Algorithm 17 Polyt0Pack(a)

Input: polynomial $a \in \mathcal{R}$

Output: byte string encoding of a

- 1: Let $b \leftarrow \lceil \log_2(q) \rceil - d$
 - 2: For each coefficient, set $t_i \leftarrow 2^{d-1} - a_i$ and pack t_i using b bits
 - 3: **return** the packed byte string
-

Algorithm 18 Polyt0Unpack(B)

Input: byte string B

Output: polynomial $a \in \mathcal{R}$

- 1: Let $b \leftarrow \lceil \log_2(q) \rceil - d$
 - 2: Unpack b -bit integers t_i and set $a_i \leftarrow 2^{d-1} - t_i$
 - 3: **return** a
-

Algorithm 19 PolyzPack(a)

Input: polynomial $a \in \mathcal{R}$ with coefficients in $\tilde{S}_{\gamma_1}$

Output: byte string encoding of a

- 1: Let $b \leftarrow \lceil \log_2(2\gamma_1) \rceil$
 - 2: For each coefficient, set $t_i \leftarrow \gamma_1 - a_i$ and pack t_i using b bits
 - 3: **return** the packed byte string
-

Algorithm 20 PolyzUnpack(B)

Input: byte string B

Output: polynomial $a \in \mathcal{R}$ with coefficients in $\tilde{S}_{\gamma_1}$

- 1: Let $b \leftarrow \lceil \log_2(2\gamma_1) \rceil$
 - 2: Unpack b -bit integers t_i and set $a_i \leftarrow \gamma_1 - t_i$
 - 3: **return** a
-

Algorithm 21 Polyw1Pack(a)

Input: polynomial $a \in \mathcal{R}_q$ **Output:** byte string encoding of $\text{HighBits}_q(a, 2\gamma_2)$

- 1: Let $b \leftarrow \lceil \log_2 \left(\left\lfloor \frac{q-1}{2\gamma_2} \right\rfloor + 1 \right) \rceil$
 - 2: Pack each coefficient $a_i \in \left\{ 0, \dots, \left\lfloor \frac{q-1}{2\gamma_2} \right\rfloor \right\}$ using b bits
 - 3: **return** the packed byte string
-

Algorithm 22 W1Encode($\mathbf{w}_1$)

Input: $\mathbf{w}_1 \in \mathcal{R}_q^k$ **Output:** byte string encoding of $\mathbf{w}_1$

- 1: $B \leftarrow \epsilon$
 - 2: **for** $i = 0$ to $k - 1$ **do**
 - 3: $B \leftarrow B \parallel \text{Polyw1Pack}(w_{1,i})$
 - 4: **end for**
 - 5: **return** B
-

Algorithm 23 HintPack($\mathbf{h}$)

Input: $\mathbf{h} \in \{0, 1\}^{k \times n}$ with total weight at most ω **Output:** byte string encoding of $\mathbf{h}$

- 1: Let $b \leftarrow \lceil \log_2(n) \rceil$
 - 2: Serialize the positions of nonzero coefficients of each polynomial in $\mathbf{h}$ using b bits per position
 - 3: Append k cumulative counts (lane bounds) so the decoder can split positions per polynomial
 - 4: Pad to a fixed length corresponding to the parameter ω
 - 5: **return** the packed byte string
-

Algorithm 24 HintUnpack(B)

Input: byte string B **Output:** $\mathbf{h} \in \{0, 1\}^{k \times n}$ or reject

- 1: Parse positions and lane bounds; reject if any position is out of range or not strictly increasing within a lane
 - 2: Reconstruct $\mathbf{h}$ and reject if the total weight exceeds ω
 - 3: **return** $\mathbf{h}$
-

These polynomial encodings are the only parameter-dependent part of key and signature formats. In particular, $(\eta, \gamma_1, \gamma_2, n)$ determine the bit-widths for $\mathbf{s}_1, \mathbf{s}_2, \mathbf{z}, \mathbf{w}_1$ and the position encoding for $\mathbf{h}$. The reference implementation instantiates these with concrete, byte-aligned layouts (e.g., packing 8 coefficients at a time for $b \in \{3, 13\}$ or 4 coefficients at a time for the Level-1 γ_1), but the mathematical specification above is generic.

3.2.3 Pseudorandom sampling

Algorithm 25 ExpandA(ρ)

```

1: function EXPANDA( $\rho$ )
2:   for  $i = 0$  to  $k - 1$  do
3:     for  $j = 0$  to  $\ell - 1$  do
4:        $a_{i,j} \leftarrow \text{SampleUniformQ}(\rho, \text{nonce} = 256 \cdot i + j)$ 
5:     end for
6:   end for
7:   return  $A = (a_{i,j}) \in \mathcal{R}_q^{k \times \ell}$ 
8: end function

```

ExpandA deterministically expands the seed ρ into a matrix whose entries are uniformly random in $\mathcal{R}_q$. Each polynomial is sampled coefficient-wise by rejection sampling from an XOF stream, ensuring uniformity in $\{0, \dots, q - 1\}$.

Algorithm 26 SampleUniformQ(ρ, nonce)

```

1: function SAMPLEUNIFORMQ( $\rho, \text{nonce}$ )
2:    $\text{ctr} \leftarrow 0, \text{block} \leftarrow 0$ 
3:   while  $\text{ctr} < n$  do
4:      $\text{in} \leftarrow \rho \parallel \text{nonce} \parallel \text{block}$ 
5:      $\text{buf} \leftarrow \text{Sample}(\text{in}, \text{outlen})$ 
6:     for each 32-bit little-endian word  $t$  parsed from  $\text{buf}$  do
7:        $t \leftarrow t \ \& \ (2^{26} - 1)$ 
8:       if  $t < q$  then
9:          $a_{\text{ctr}} \leftarrow t; \text{ctr} \leftarrow \text{ctr} + 1$ 
10:        if  $\text{ctr} = n$  then
11:          break
12:        end if
13:      end if
14:    end for
15:     $\text{block} \leftarrow \text{block} + 1$ 
16:  end while
17:  return  $a \in \mathcal{R}_q$  with coefficients in  $\{0, \dots, q - 1\}$ 
18: end function

```

SampleUniformQ uses rejection sampling to ensure uniform coefficients modulo q . For the fixed modulus $q = 67104769 < 2^{26}$, it is efficient to mask candidate values to 26 bits and accept those below q .

Algorithm 27 SampleEta(ρ', nonce)

```

Input: seed  $\rho'$ , nonce  $\text{nonce}$ 
Output:  $s \in \mathcal{R}$  with  $\|s\|_\infty \leq \eta$ 
1: Use  $\text{Sample}(\rho' \parallel \text{nonce}, \text{outlen})$  to obtain a pseudorandom byte stream
2: Parse bytes and map to coefficients in  $[-\eta, \eta]$  using a parameter-dependent rejection method
3: return  $s \in \mathcal{R}$  with  $\|s\|_\infty \leq \eta$ 

```

Algorithm 28 SampleGamma1(ρ'' , nonce)

Input: seed ρ'' , nonce nonce

Output: $y \in \mathcal{R}$ with coefficients in $\tilde{S}_{\gamma_1}$

- 1: Let $w := \lceil \log_2(2\gamma_1) \rceil$
 - 2: Use Sample($\rho'' \parallel \text{nonce}, \text{outlen}$) to obtain enough bits
 - 3: **for** $i = 0$ to $n - 1$ **do**
 - 4: Interpret the next w bits as an integer $t \in \{0, \dots, 2^w - 1\}$
 - 5: Set $y_i := \gamma_1 - t$
 - 6: **end for**
 - 7: **return** $y \in \mathcal{R}$ with coefficients in $\tilde{S}_{\gamma_1}$
-

SampleEta and SampleGamma1 produce short polynomials from seeds and nonces. The exact parsing and rejection rules depend on η and γ_1 . In particular, different instances use different (η, γ_1) values; therefore packing and sampling are parameterized rather than hard-coded in this specification. SampleInBall deterministically maps the 256-bit seed $\tilde{c}$ to a sparse challenge polynomial c of Hamming weight τ , ensuring that challenges are uniformly distributed over $\mathcal{B}_\tau$.

3.2.4 High-order and low-order bits and hints

Algorithm 29 Power2Round(a)

Input: $a \in \mathbb{Z}_q$

Output: (a_1, a_0) with $a \equiv a_1 \cdot 2^d + a_0 \pmod{q}$

- 1: $a \leftarrow a \bmod^+ q$
 - 2: $a_1 \leftarrow \left\lfloor \frac{a + 2^{d-1} - 1}{2^d} \right\rfloor$
 - 3: $a_0 \leftarrow a - a_1 \cdot 2^d$
 - 4: **return** (a_1, a_0)
-

Algorithm 30 Decompose(a)

Input: $a \in \mathbb{Z}_q$

Output: (a_1, a_0) with respect to $\alpha = 2\gamma_2$

- 1: $a \leftarrow a \bmod^+ q$
 - 2: $\alpha \leftarrow 2\gamma_2$
 - 3: $a_1 \leftarrow \left\lfloor \frac{a + \gamma_2}{\alpha} \right\rfloor$
 - 4: If $a_1 = \left\lceil \frac{q}{\alpha} \right\rceil$, set $a_1 \leftarrow 0$
 - 5: $a_0 \leftarrow a - a_1 \cdot \alpha$
 - 6: If $a_0 > (q - 1)/2$, set $a_0 \leftarrow a_0 - q$
 - 7: **return** (a_1, a_0)
-

Algorithm 31 MakeHint(a_0, a_1)

Input: (a_0, a_1) from Decompose

Output: hint bit $h \in \{0, 1\}$

- 1: **if** $a_0 > \gamma_2$ **or** $a_0 < -\gamma_2$ **or** $(a_0 = -\gamma_2 \text{ and } a_1 \neq 0)$ **then**
 - 2: **return** 1
 - 3: **else**
 - 4: **return** 0
 - 5: **end if**
-

Algorithm 32 UseHint(a, h)

Input: $a \in \mathbb{Z}_q$, hint bit $h \in \{0, 1\}$
Output: adjusted high bits $a'_1 \in \{0, \dots, \lceil q/(2\gamma_2) \rceil - 1\}$

- 1: Compute $(a_1, a_0) \leftarrow \text{Decompose}(a)$
- 2: **if** $h = 0$ **then**
- 3: **return** a_1
- 4: **end if**
- 5: $m \leftarrow \left\lceil \frac{q}{2\gamma_2} \right\rceil$
- 6: **if** $a_0 > 0$ **then**
- 7: **return** $(a_1 + 1) \bmod m$
- 8: **else**
- 9: **return** $(a_1 - 1) \bmod m$
- 10: **end if**

Power2Round is used in key generation to publish only a rounded version $\mathbf{t}_1$ while keeping $\mathbf{t}_0$ secret, with $\mathbf{t} \equiv \mathbf{t}_1 \cdot 2^d + \mathbf{t}_0 \pmod{q}$ coefficient-wise. Decompose is used in signing and verification to define $\text{HighBits}_q(\cdot, 2\gamma_2)$ and $\text{LowBits}_q(\cdot, 2\gamma_2)$: for each coefficient a , set $(a_1, a_0) = \text{Decompose}(a)$ and define $\text{HighBits}_q(a, 2\gamma_2) := a_1$ and $\text{LowBits}_q(a, 2\gamma_2) := a_0$. The hint mechanism (MakeHint/UseHint) ensures that the verifier reconstructs the same high bits as the signer under the bounded perturbations enforced by the rejection conditions in Algorithm 2.

3.2.5 NTT and inverse NTT

Algorithm 33 NTT($a_0, \dots, a_{n-1}$)

Input: coefficients $(a_0, \dots, a_{n-1})$ representing $a \in \mathcal{R}_q$
Output: NTT-domain representation of a

- 1: $k \leftarrow 1$
- 2: **for** $\text{len} \leftarrow n/2$ down to 1 halving each step **do**
- 3: **for** $\text{start} \leftarrow 0$ to $n - 1$ stepping by $2 \cdot \text{len}$ **do**
- 4: $\zeta \leftarrow \text{zetas}[k]$; $k \leftarrow k + 1$
- 5: **for** $j \leftarrow \text{start}$ to $\text{start} + \text{len} - 1$ **do**
- 6: $t \leftarrow \text{MontgomeryReduce}(\zeta \cdot a_{j+\text{len}})$
- 7: $a_{j+\text{len}} \leftarrow a_j - t \pmod{q}$
- 8: $a_j \leftarrow a_j + t \pmod{q}$
- 9: **end for**
- 10: **end for**
- 11: **end for**
- 12: **return** $(a_0, \dots, a_{n-1})$

Algorithm 34 $\text{InvNTT}(\hat{a}_0, \dots, \hat{a}_{n-1})$

Input: NTT-domain coefficients $(\hat{a}_0, \dots, \hat{a}_{n-1})$

Output: coefficients of $a \in \mathcal{R}_q$

```
1:  $k \leftarrow n - 1$ 
2: for  $\text{len} \leftarrow 1$  up to  $n/2$  doubling each step do
3:   for  $\text{start} \leftarrow 0$  to  $n - 1$  stepping by  $2 \cdot \text{len}$  do
4:      $\zeta \leftarrow \text{zetas}[k]; k \leftarrow k - 1$ 
5:     for  $j \leftarrow \text{start}$  to  $\text{start} + \text{len} - 1$  do
6:        $t \leftarrow \hat{a}_j$ 
7:        $\hat{a}_j \leftarrow t + \hat{a}_{j+\text{len}} \pmod{q}$ 
8:        $\hat{a}_{j+\text{len}} \leftarrow t - \hat{a}_{j+\text{len}} \pmod{q}$ 
9:        $\hat{a}_{j+\text{len}} \leftarrow \text{MontgomeryReduce}(\zeta \cdot \hat{a}_{j+\text{len}})$ 
10:    end for
11:  end for
12: end for
13: Multiply all coefficients by  $n^{-1} \pmod{q}$  (in Montgomery form)
14: return  $(\hat{a}_0, \dots, \hat{a}_{n-1})$ 
```

The reference implementation uses a radix-2 NTT over $\mathcal{R}_q$ with a fixed modulus q and precomputed twiddle factors (“zetas”). The NTT enables fast polynomial multiplication in $\mathcal{R}_q$ and is used in key generation, signing, and verification.

3.2.6 Parallel small polynomial multiplications

Algorithm 35 SmallPoly($c, \mathbf{a}, U, r$)

Input: A polynomial $c = \sum_{i=0}^{n-1} c_i x^i \in B_\tau$ and a polynomial vector $\mathbf{a} = (a^{(0)}, \dots, a^{(r-1)}) \in R_q^r$, where $a^{(j)} = \sum_{i=0}^{n-1} a_i^{(j)} x^i \in R_q$ and $a_i^{(j)} \in \{-U, \dots, U\}$ for all $i \in \{0, \dots, n-1\}$ and $j \in \{0, \dots, r-1\}$. The coefficient bound U , and the vector dimension r .

Output: $\mathbf{u} = [u^{(0)}, \dots, u^{(r-1)}]^T \in R_q^r$, where $u^{(j)} = c \cdot a^{(j)} \in R_q$.

```

1: for  $i = 0$  to  $n - 1$  do
2:    $w_i \leftarrow 0$ 
3:    $v_i \leftarrow 0$ 
4:    $v_{i-n} \leftarrow 0$ 
5:   for  $j = 0$  to  $r - 1$  do
6:      $v_i \leftarrow v_i \cdot M + (U + a_i^{(j)})$ 
7:      $v_{i-n} \leftarrow v_{i-n} \cdot M + (U - a_i^{(j)})$ 
8:   end for
9: end for
10:  $\zeta \leftarrow 2U \cdot \frac{M^r - 1}{M - 1}$ 
11: for  $i = 0$  to  $n - 1$  do
12:   if  $c_i = 1$  then
13:     for  $j = 0$  to  $n - 1$  do
14:        $w_j \leftarrow w_j + v_{j-i}$ 
15:     end for
16:   end if
17:   if  $c_i = -1$  then
18:     for  $j = 0$  to  $n - 1$  do
19:        $w_j \leftarrow w_j + (\zeta - v_{j-i})$ 
20:     end for
21:   end if
22: end for
23: for  $i = 0$  to  $n - 1$  do
24:    $t \leftarrow w_i$ 
25:   for  $j = 0$  to  $r - 1$  do
26:      $u_i^{(r-1-j)} \leftarrow (t \bmod M) - \tau U \pmod{q}$ 
27:      $t \leftarrow \lfloor t/M \rfloor$ 
28:   end for
29: end for
30: for  $j = 0$  to  $r - 1$  do
31:    $u^{(j)} \leftarrow \sum_{i=0}^{n-1} u_i^{(j)} x^i$ 
32: end for
33:  $\mathbf{u} \leftarrow [u^{(0)}, \dots, u^{(r-1)}]^T$ 
34: return  $\mathbf{u}$ 

```

$$\triangleright M = 2^{\log_2(1+2\tau U)}$$

Besides multiplications between polynomials of standard forms, OPS-SIG frequently requires computing products of the form $c\mathbf{a}$, where c is a sparse small-coefficient polynomial with fixed Hamming weight and coefficients in $0, 1$, and $\mathbf{a}$ is a polynomial vector. Owing to the sparsity and simple coefficient structure of c , it is possible to exploit these properties to construct algebraic parallelism and transform the multiplication into a sequence of additions, subtractions, and cyclic shifts, thereby significantly reducing the computational overhead [5].

Algorithm 35 describes the basic parallel small polynomial multiplication procedure. The

algorithm takes as input a small-coefficient polynomial c , a polynomial vector $\mathbf{a}$, the coefficient bound of $\mathbf{a}$, and the vector dimension r . Then it outputs the product $\mathbf{ca}$.

Algorithm 36 SmallPolywithEarlycheck($\mathbf{z}, \mathbf{w}, c, \mathsf{T}_{\mathbf{s}_1}, \mathsf{T}_{\mathbf{s}_2}, \mathbf{w}_0, \mathbf{y}, B_1, B_2$)

```

1: Input:  $c \in B_\tau$ , lookup tables  $\mathsf{T}_{\mathbf{s}_1}, \mathsf{T}_{\mathbf{s}_2}$ , polynomial vectors  $\mathbf{y} \in \mathcal{R}_q^\ell$ ,  $\mathbf{w} \in \mathcal{R}_q^k$ , bounds  $B_1, B_2$ .
2: Output: 0 if accepted, 1 if rejection is triggered.
3: Initialize arrays:  $\mathbf{answer}[0..n-1] \leftarrow 0$ ,  $\mathbf{answer2}[0..n-1] \leftarrow 0$ 
4:  $M_1 = 2^{\log_2(1+2\tau\eta_1)}$ ,  $M_2 = 2^{\log_2(1+2\tau\eta_2)}$ 
5:  $\zeta_1 \leftarrow 2\eta_1 \cdot \frac{M_1^\ell - 1}{M_1 - 1}$ ,  $\zeta_2 \leftarrow 2\eta_2 \cdot \frac{M_2^k - 1}{M_2 - 1}$ 
6: for  $i = 0$  to  $n - 1$  do
7:   if  $c_i = 1$  then
8:     for  $j = 0$  to  $n - 1$  do
9:        $\mathbf{answer}[j] += \mathsf{T}_{\mathbf{s}_1}[j - i + n]$ 
10:       $\mathbf{answer2}[j] += \mathsf{T}_{\mathbf{s}_2}[j - i + n]$ 
11:    end for
12:   else if  $c_i = -1$  then
13:     for  $j = 0$  to  $n - 1$  do
14:        $\mathbf{answer}[j] += \zeta_1 - \mathsf{T}_{\mathbf{s}_1}[j - i + n]$ 
15:        $\mathbf{answer2}[j] += \zeta_2 - \mathsf{T}_{\mathbf{s}_2}[j - i + n]$ 
16:     end for
17:   end if
18: end for
19: for  $i = 0$  to  $n - 1$  do
20:    $t_1 \leftarrow \mathbf{answer}[i]$ 
21:    $t_2 \leftarrow \mathbf{answer2}[i]$ 
22:   for  $j = 0$  to  $\ell - 1$  do
23:      $\mathbf{z}_i^{(\ell-1-j)} \leftarrow (t_1 \bmod M_1) - \tau\eta_1$ 
24:      $\mathbf{z}_i^{(\ell-1-j)} \leftarrow \mathbf{z}_i^{(\ell-1-j)} + \mathbf{y}_i^{(\ell-1-j)}$ 
25:      $t_1 \leftarrow \lfloor t_1 / M_1 \rfloor$ 
26:     if  $\mathbf{z}_i^{(\ell-1-j)} \geq B_1$  or  $\mathbf{z}_i^{(\ell-1-j)} \leq -B_1$  then
27:       return 1
28:     end if
29:   end for
30:   for  $j = 0$  to  $k - 1$  do
31:      $\mathbf{w}_i^{(k-1-j)} \leftarrow (t_2 \bmod M_2) - \tau\eta_2$ 
32:      $\mathbf{w}_i^{(k-1-j)} \leftarrow \mathbf{w}_i^{(k-1-j)} - \mathbf{w}_0^{(k-1-j)}$ 
33:      $t_2 \leftarrow \lfloor t_2 / M_2 \rfloor$ 
34:     if  $\mathbf{w}_i^{(k-1-j)} \geq B_2$  or  $\mathbf{w}_i^{(k-1-j)} \leq -B_2$  then
35:       return 1
36:     end if
37:   end for
38: end for
39: return 0

```

Algorithm 37 PrepareTable($\mathbf{a}, U, r$)

1: **Input:** polynomial vector $\mathbf{a}_1 \in \mathcal{R}_q^r$, the coefficient bound U , and the vector dimension r .
Ensure: Precomputed table $\mathbf{T}_\mathbf{a}$
2: **for** $i = 0$ to $n - 1$ **do**
3: $v_i \leftarrow 0$
4: $v_{i-n} \leftarrow 0$
5: **for** $j = 0$ to $r - 1$ **do**
6: $v_i \leftarrow v_i \cdot M + (U + a_i^{(j)})$ $\triangleright M = 2^{\log_2(1+2\tau U)}$
7: $v_{i-n} \leftarrow v_{i-n} \cdot M + (U - a_i^{(j)})$
8: **end for**
9: **end for**
10: **for** $i = 0$ to $2n - 1$ **do**
11: $\mathbf{T}_\mathbf{a}[i] = v_{i-n}$
12: **end for**

The basic small polynomial multiplication can be combined with rejection sampling in the Fiat-Shamir-with-abort paradigm, thereby accelerating the multiplication computation while reducing the computational cost incurred by rejection sampling. Algorithm 36 describes this process, and Algorithm 37 provides the preprocessing steps required for using Algorithm 36.

3.2.7 Arithmetic under NTT

Algorithm 38 MontgomeryReduce(x)

Input: integer x
Output: $x \cdot R^{-1} \bmod q$ for fixed radix $R = 2^{32}$
1: Compute $x \cdot R^{-1} \bmod q$ for a fixed radix $R = 2^{32}$ without division
2: **return** the reduced value in $\mathbb{Z}_q$

Algorithm 39 PointwiseMontgomery($\hat{a}, \hat{b}$)

Input: NTT-domain polynomials $\hat{a}, \hat{b} \in \mathbb{Z}_q^n$
Output: NTT-domain product $\hat{c}$
1: **for** $i = 0$ to $n - 1$ **do**
2: $\hat{c}_i \leftarrow \text{MontgomeryReduce}(\hat{a}_i \cdot \hat{b}_i)$
3: **end for**
4: **return** $\hat{c}$

NTT-domain multiplication reduces polynomial multiplication to pointwise multiplication followed by an inverse NTT. Montgomery reduction is used to keep intermediate products bounded and efficient on standard 32/64-bit platforms.

4 Design Rationale

4.1 Underlying hardness assumptions

OPS-SIG is designed around Module-LWE (MLWE) and Module-SIS (MSIS) hardness assumptions, using a module setting to reduce public key and signature sizes, following the design methodology of [1].

4.2 Parameter selection

Parameters are chosen to meet the claimed classical and quantum security strengths while keeping key and signature sizes compact. The core design tradeoffs are:

- Increasing (k, ℓ) increases security margins at the cost of larger keys and higher cost in matrix-vector multiplications.
- Increasing η and adjusting (γ_1, γ_2) affects rejection sampling rate, signing time, and the size of packed secrets and signature responses.
- Increasing n (Level-5) increases security and cost; it also changes NTT sizes and packing formats.

4.3 Failure probability

The signing algorithm uses rejection sampling. The algorithm repeats until all norm bounds and the hint-weight bound are met, yielding a negligible probability of outputting an invalid signature. The expected number of iterations is reported in Table 1 (column “Rep.”). In the reference implementation, signing never returns failure; it loops until success.

5 Security Statement and Analysis

5.1 Security claims

OPS-SIG is designed to achieve strong unforgeability under adaptive chosen-message attacks (SUF-CMA) in the (Quantum) random oracle model, following the Fiat-Shamir-with-aborts paradigm [1]. The security reductions are based on the hardness of MSIS/MLWE problems as in [1]. We additionally provide a QROM discussion based on standard “measure-and-reprogram” style arguments; the resulting bounds are weaker but support the claim that OPS-SIG contains no component known to be fundamentally broken by quantum algorithms beyond the usual quantum speedups for generic search.

5.2 Security model

A signature scheme $\text{SIG} = (\text{KeyGen}, \text{Sign}, \text{Verify})$ is said to be SUF-CMA secure if for any probabilistic polynomial-time (PPT) adversary $\mathcal{A}$, the advantage defined by the following experiment is negligible in the security parameter λ :

$$\mathbf{Adv}_{\text{SIG}, \mathcal{A}}^{\text{SUF-CMA}}(\lambda) = \Pr \left[\begin{array}{l} (sk, pk) \leftarrow \text{KeyGen}(); \\ b = 1 : (m^*, \sigma^*) \leftarrow A^{\mathcal{O}_{\text{Sign}(\cdot)}}(pk); \\ b := \text{Verify}(pk, m^*, \sigma^*) \end{array} \right]$$

On input a message $m \in \{0, 1\}^*$, the oracle $\mathcal{O}_{\text{Sign}}(sk, m)$ returns a signature $\sigma \leftarrow \text{Sign}(sk, m)$ and adds the pair (m, σ) to the set $\mathcal{Q}$. In the SUF-CMA setting, the pair (m^*, σ^*) output by $\mathcal{A}$ must satisfy $(m^*, \sigma^*) \notin \mathcal{Q}$.

5.3 Hardness assumptions

We use the following problems over $\mathcal{R}_q = \mathbb{Z}_q[X]/(X^n + 1)$ and module ranks (k, ℓ) .

The Decisional MLWE Problem. For integers l, k , and a probability distribution $\mathcal{D} : \mathcal{R}_q \rightarrow [0, 1]$. Given an uniformly distributed matrix $A \in \mathcal{R}_q^{k \times l}$ and a vector $\mathbf{t}$ distributed according to either $\mathbf{t} \in \mathcal{D}^k$, or $\mathbf{t} = A\mathbf{s}_1 + \mathbf{s}_2$ ($\mathbf{s}_1 \in \mathcal{D}^l, \mathbf{s}_2 \in \mathcal{D}^k$), distinguish which is the case. The advantage of adversary $\mathcal{A}$ in solving the decisional MLWE problem is

$$\text{Adv}_{k,l,\mathcal{D}}^{\text{MLWE}}(\mathcal{A}) := \left| \Pr \left[b = 1 : \begin{array}{l} A \leftarrow \mathcal{R}_q^{k \times l}, \\ \mathbf{t} \leftarrow \mathcal{D}^k; \\ b \leftarrow \mathcal{A}(A, \mathbf{t}) \end{array} \right] - \Pr \left[b = 1 : \begin{array}{l} A \leftarrow \mathcal{R}_q^{k \times l}, \\ \mathbf{s}_1 \leftarrow \mathcal{D}^l, \mathbf{s}_2 \leftarrow \mathcal{D}^k; \\ b \leftarrow \mathcal{A}(A, A\mathbf{s}_1 + \mathbf{s}_2) \end{array} \right] \right|$$

The hardness of the decisional MLWE problem means that $\text{Adv}_{k,l,\mathcal{D}}^{\text{MLWE}}(\mathcal{A})$ is negligible for all probabilistic polynomial-time $\mathcal{A}$.

The MSIS Problem. Given an uniformly distributed matrix $A \in \mathcal{R}_q^{k \times l}$, find a nonzero vector $\mathbf{y} \in \mathcal{R}^{k+l}$ of norm $\|\mathbf{y}\|_\infty \leq \gamma$ such that $[\mathbf{I}|A] \cdot \mathbf{y} = \mathbf{0}$. The advantage of adversary $\mathcal{A}$ in solving the (Hermite Normal Form) MSIS problem is

$$\text{Adv}_{k,l,\gamma}^{\text{MSIS}}(\mathcal{A}) := \Pr \left[0 < \|\mathbf{y}\|_\infty \leq \gamma \wedge [\mathbf{I}|A] \cdot \mathbf{y} = \mathbf{0} : \begin{array}{l} A \leftarrow \mathcal{R}_q^{k \times l}, \\ \mathbf{y} \leftarrow \mathcal{A}(A) \end{array} \right]$$

The hardness of the MSIS problem means that $\text{Adv}_{k,l,\gamma}^{\text{MSIS}}(\mathcal{A})$ is negligible for all probabilistic polynomial-time $\mathcal{A}$.

The SelfTargetMSIS Problem. Given an uniformly distributed matrix $A \in \mathcal{R}_q^{k \times l}$ and a cryptographic hash function $H : \{0, 1\} \rightarrow \mathcal{B}_\tau$, find a nonzero vector $\mathbf{y} = [\mathbf{r}^T | c]^T \in \mathcal{R}^{k+l}$ of norm $\|\mathbf{y}\|_\infty \leq \gamma$ and μ such that $H(\mu \parallel [\mathbf{I}|A] \cdot \mathbf{y}) = c$. The advantage of adversary $\mathcal{A}$ in solving the SelfTargetMSIS problem is

$$\text{Adv}_{H,k,l,\gamma}^{\text{SelfTargetMSIS}}(\mathcal{A}) := \Pr \left[0 < \|\mathbf{y}\|_\infty \leq \gamma \wedge H(\mu \parallel [\mathbf{I}|A] \cdot \mathbf{y}) = c : \begin{array}{l} A \leftarrow \mathcal{R}_q^{k \times l}, \\ (\mathbf{y} := \begin{bmatrix} \mathbf{r} \\ c \end{bmatrix}, \mu) \leftarrow \mathcal{A}^{H(\cdot)}(A) \end{array} \right]$$

The hardness of the SelfTargetMSIS problem means that $\text{Adv}_{H,k,l,\gamma}^{\text{SelfTargetMSIS}}(\mathcal{A})$ is negligible for all probabilistic polynomial-time $\mathcal{A}$. SelfTargetMSIS is based on the combined hardness of MSIS and the hash function H . There is a (non-tight) reduction from MSIS to SelfTargetMSIS in the classical ROM [6].

5.4 Theoretical security in the ROM

Theorem 2 (SUF-CMA security in the ROM). *In the Random Oracle Model (ROM), let $H(\rho) = \text{SampleInBall}(H_{2\lambda}(\mu \parallel \text{W1Encode}(\mathbf{w}_1)))$, where $H : \{0, 1\}^* \rightarrow \mathcal{B}_\tau$ and the pseudorandom samplings are modeled as random oracles. Assuming CRH is a collision-resistant hash function, OPS-SIG is SUF-CMA secure under the MLWE and MSIS assumptions.*

In the following security proof, we will assume that the public key of OPS-SIG obtained by the adversary is $(\rho, \mathbf{t})$. In the actual OPS-SIG, the public key only contains the seed ρ and the high bits $\mathbf{t}_1$ of $\mathbf{t} = A\mathbf{s}_1 + \mathbf{s}_2$; therefore, from an attacking perspective, this assumption is actually more beneficial to the adversary, while from a security perspective, this assumption also means it might be more difficult for the adversary to attack the SUF-CMA security of OPS-SIG in practice. In addition, the fact that $\mathbf{t}_0$ is not part of the actual public key does not affect any steps of the proof. Therefore, this assumption is reasonable and provides more conservative security.

The proof of Theorem 2 is divided into two stages. The first stage proves that the behavior of the signing oracle is statistically indistinguishable from that of an efficient PPT simulator. The second stage proves that, assuming the hardness of the underlying problems, no efficient attacker can forge a valid message-signature pair with non-negligible probability. These stages are formally established in Lemma 1 and Lemma 2, respectively.

Lemma 1. *Assuming H is a random oracle, then there exists an efficient simulator $\mathcal{S}$ that can successfully simulate the signing oracle $\mathcal{O}_{\text{Sign}}(\cdot)$ knowing only the public key $pk = (\rho, \mathbf{t})$.*

Proof. In the actual signature algorithm, the following probability equation holds:

$$\Pr[\mathbf{z}, c] = \Pr[c] \cdot \Pr[\mathbf{y} = \mathbf{z} - c\mathbf{s}_1 | c]$$

where the probability comes from the random sampling of $\mathbf{y}$ and the random oracle H . When the signature satisfies $\|\mathbf{z}\|_\infty < \gamma_1 - \beta_1$, since when $\|(c\mathbf{s}_1)\|_\infty \leq \beta_1$, we have $\|(\mathbf{y})\|_\infty = \|(\mathbf{z} - c\mathbf{s}_1)\|_\infty \leq \gamma_1 - 1$, any $\mathbf{y}$ sampled according to the signature algorithm rules satisfies this condition. Therefore, if the rejection sampling directly outputs $\mathbf{z}$ satisfying $\|(\mathbf{z})\|_\infty < \gamma_1 - \beta_1$, the joint distribution of $(\mathbf{z}, c)$ is equivalent to a random distribution on $S_{\gamma_1 - \beta_1 - 1} \times B_\tau$. Based on the above probability analysis, it is easy to construct a simulator $\mathcal{S}$ that knows only $pk = (\rho, \mathbf{t})$ to simulate the signing oracle $\mathcal{O}_{\text{Sign}}(\cdot)$. The simulated signing process of $\mathcal{S}$ is as follows: When performing a simulated signature, simulator $\mathcal{S}$ randomly selects $(\mathbf{z}, c)$ from the aforementioned distribution space, and outputs $(\mathbf{z}, c)$ if and only if the following inequality is satisfied:

$$\text{LowBits}((A\mathbf{z} - c\mathbf{t}), 2\gamma_2)_\infty < \gamma_2 - \beta_2.$$

Since $\mathbf{w} - c\mathbf{s}_2 = A\mathbf{z} - c\mathbf{t}$ holds, the following inequality naturally holds:

$$\|(\mathbf{r}_0)\|_\infty := \|\text{LowBits}((\mathbf{w} - c\mathbf{s}_2), 2\gamma_2)\|_\infty < \gamma_2 - \beta_2.$$

Since $\|(c\mathbf{s}_2)\|_\infty \leq \beta_2$, according to Lemma 2 in [6], at this time we have:

$$\mathbf{r}_1 = \text{HighBits}(\mathbf{w} - c\mathbf{s}_2, 2\gamma_2) = \text{HighBits}(\mathbf{w}, 2\gamma_2) = \mathbf{w}_1$$

Therefore, simulator $\mathcal{S}$ does not need to check whether $\mathbf{r}_1$ equals $\mathbf{w}_1$ but can always assume it holds. Subsequently, $\mathcal{S}$ simply programs the random oracle as $H(\mu \parallel \mathbf{w}_1) = c$; this step may fail only if the result of $H(\mu \parallel \mathbf{w}_1)$ was previously designated as another value, and the failure probability is negligible. At this point, the simulator has obtained $(\mathbf{z}, c)$ satisfying the pre-calculation process and the rejection sampling conditions $\|(\mathbf{z})\|_\infty < \gamma_1 - \beta_1$ and $\|(\mathbf{r}_0)\|_\infty < \gamma_2 - \beta_2$. Subsequently, the simulator can further execute the subsequent rejection sampling process and calculate without using the private key:

$$\mathbf{h} := \text{MakeHint}(-c\mathbf{t}_0, \mathbf{w} - c\mathbf{s}_2 + c\mathbf{t}_0) = \text{MakeHint}(-c\mathbf{t}_0, A\mathbf{z} - c\mathbf{t}_1 \cdot 2^d)$$

In summary, the simulator can successfully simulate the signature $(\mathbf{z}, \mathbf{h}, c)$ following the true distribution. $\square$

Lemma 2. *If there exists a PPT adversary $\mathcal{A}$ that can forge a valid OPS-SIG signature, then there also exists an extractor $\mathcal{E}$ that can solve the MSIS problem. Specifically, for a randomly sampled matrix $A \leftarrow \mathcal{R}_q^{k \times l}$ and vector $\mathbf{t} \leftarrow \mathcal{R}_q^k$, one can find $\mathbf{u}_1 \in \mathcal{R}_q^l, \mathbf{u}_2 \in \mathcal{R}_q^l, \mathbf{u}_3 \in \mathcal{R}_q^l$ satisfying: $\|\mathbf{u}_1\|_\infty \leq 2(\gamma_1 - \beta_1)$, $\|\mathbf{u}_2\|_\infty \leq 4\gamma_2 + 2$, $\|\mathbf{u}_3\|_\infty \leq 2$, $A\mathbf{u}_1 + \mathbf{u}_2 = \mathbf{u}_3\mathbf{t}_1 \cdot 2^d$, $\mathbf{u}_1, \mathbf{u}_2, \mathbf{u}_3 \neq 0$, $\mathbf{u}_2$ has at most 2ω coefficients with absolute values greater than $2\gamma_2$.*

Proof. Lemma 2 is proven by constructing an extractor $\mathcal{E}$ capable of solving the MSIS problem. Extractor $\mathcal{E}$ knows $(A, \mathbf{t})$; at this point, $\mathcal{E}$ can randomly select a string ρ , and program the random oracle such that $A \leftarrow \mathcal{R}_q^{k \times l} := \text{ExpandA}(\rho)$ (based on the properties of the random

oracle and the randomness of ρ , the probability that ρ was previously used to query the random oracle $\text{ExpandA}(\rho)$ is negligible), then outputs the public key $pk = (\rho, \mathbf{t})$, and uses the simulator $\mathcal{S}$ in Lemma 1 to answer signature oracle queries from adversary $\mathcal{A}$. If adversary $\mathcal{A}$ successfully forges a valid signature $(\mathbf{z}, \mathbf{h}, c)$ for message M , let $\mu := \text{CRH}(\text{CRH}(\rho \parallel \mathbf{t}_1) \parallel M)$, then the following relationship is satisfied:

$$H(\mu \parallel \mathbf{w}_1 = \text{UseHint}(\mathbf{h}, A\mathbf{z} - c\mathbf{t}_1 \cdot 2^d, 2\gamma_2)) = c$$

Below we consider two different cases.

- **Case 1.** If c was produced by the signature oracle, then $\mathcal{S}$ has already answered a signature $(\mathbf{z}', \mathbf{h}', c)$ for some message M' . According to the strong unforgeability success condition, we have $(M', (\mathbf{z}', \mathbf{h}', c)) \neq (M, (\mathbf{z}, \mathbf{h}, c))$. Let $\mu' := \text{CRH}(\text{CRH}(\rho \parallel \mathbf{t}_1) \parallel M')$, then there exists $\mathbf{w}'_1$ such that the equation $H(\mu \parallel \mathbf{w}_1) = H(\mu' \parallel \mathbf{w}'_1) = c$ holds. According to the properties of the random oracle H , the equations $\mu = \mu'$ and $\mathbf{w}_1 = \mathbf{w}'_1$ hold with probability $1 - \text{negl}(1^\lambda)$. According to the collision-resistant property of CRH , we have $M \neq M'$. Therefore, we have

$$\mathbf{w}_1 = \text{UseHint}(\mathbf{h}, A\mathbf{z} - c\mathbf{t}_1 \cdot 2^d, 2\gamma_2)$$

$$\mathbf{w}_1 = \text{UseHint}(\mathbf{h}', A\mathbf{z}' - c\mathbf{t}_1 \cdot 2^d, 2\gamma_2)$$

If $\mathbf{z} = \mathbf{z}'$, it follows from Lemma 1 in [6] that $\mathbf{h} = \mathbf{h}'$, which contradicts our assumption. Therefore, we must be $\mathbf{z} \neq \mathbf{z}'$. According to Lemma 1 in [6], the following relationships hold:

$$\|A\mathbf{z} - c\mathbf{t}_1 \cdot 2^d - \mathbf{w}_1 \cdot 2\gamma_2\|_\infty \leq 2\gamma_2 + 1$$

$$\|A\mathbf{z}' - c\mathbf{t}_1 \cdot 2^d - \mathbf{w}_1 \cdot 2\gamma_2\|_\infty \leq 2\gamma_2 + 1$$

From the triangle inequality, it is known that $\|A(\mathbf{z} - \mathbf{z}')\|_\infty \leq 4\gamma_2 + 2$. At this point, there exist $\mathbf{u}_1 \in \mathcal{R}_q^l, \mathbf{u}_2 \in \mathcal{R}_q^k, u_3 \in \mathcal{R}_q$ satisfying $\|\mathbf{u}_1\|_\infty \leq 2(\gamma_1 - \beta_1), \|\mathbf{u}_2\|_\infty \leq 4\gamma_2 + 2$, and $A\mathbf{u}_1 + \mathbf{u}_2 = \mathbf{0}$, where $\mathbf{u}_1 = (\mathbf{z} - \mathbf{z}') \neq \mathbf{0}$. Since in valid signatures, $\mathbf{h}$ and $\mathbf{h}'$ have at most ω non-zero entries, according to Lemma 1 in [6], the vectors $A\mathbf{z} - c\mathbf{t}_1 \cdot 2^d - \mathbf{w}_1 \cdot 2\gamma_2$ and $A\mathbf{z}' - c\mathbf{t}_1 \cdot 2^d - \mathbf{w}_1 \cdot 2\gamma_2$ have at most ω coefficients with absolute values greater than γ_2 . Therefore, the vector $\mathbf{u}_2$ has at most 2ω coefficients with absolute values greater than $2\gamma_2$. In summary, extractor $\mathcal{E}$ has solved the MSIS problem in the lemma.

- **Case 2.** If c was produced by adversary $\mathcal{A}$ querying the random oracle H , i.e., $H(\mu' \parallel \mathbf{w}'_1) = c$. According to the properties of the random oracle, the equations $\mu = \mu' = \text{CRH}(\text{CRH}(\rho \parallel \mathbf{t}_1) \parallel M)$ and $\mathbf{w}_1 = \mathbf{w}'_1 = \text{UseHint}(\mathbf{h}, A\mathbf{z} - c\mathbf{t}_1 \cdot 2^d, 2\gamma_2)$ hold with probability $1 - \text{negl}(1^\lambda)$. According to the standard Forking Lemma, $\mathcal{E}$ can extract two forged signatures $(\mathbf{z}, \mathbf{h}, c)$ and $(\mathbf{z}', \mathbf{h}', c')$ satisfying $c \neq c'$ and:

$$\mathbf{w}_1 = \text{UseHint}(\mathbf{h}, A\mathbf{z} - c\mathbf{t}_1 \cdot 2^d, 2\gamma_2)$$

$$\mathbf{w}'_1 = \text{UseHint}(\mathbf{h}', A\mathbf{z}' - c'\mathbf{t}_1 \cdot 2^d, 2\gamma_2)$$

According to Lemma 1 in [6], the following relationships hold:

$$\|A\mathbf{z} - c\mathbf{t}_1 \cdot 2^d - \mathbf{w}_1 \cdot 2\gamma_2\|_\infty \leq 2\gamma_2 + 1$$

$$\|A\mathbf{z}' - c'\mathbf{t}_1 \cdot 2^d - \mathbf{w}_1 \cdot 2\gamma_2\|_\infty \leq 2\gamma_2 + 1$$

From the triangle inequality, it is known that $\|A(\mathbf{z} - \mathbf{z}') - (c - c')\mathbf{t}_1 \cdot 2^d\|_\infty \leq 4\gamma_2 + 2$. At this point, there exist $\mathbf{u}_1 \in \mathcal{R}_q^l, \mathbf{u}_2 \in \mathcal{R}_q^k, u_3 \in \mathcal{R}_q$ satisfying $\|\mathbf{u}_1\|_\infty \leq 2(\gamma_1 - \beta_1), \|\mathbf{u}_2\|_\infty \leq$

$4\gamma_2 + 2, \|u_3\|_\infty \leq 2$, where $\mathbf{u}_1 = (\mathbf{z} - \mathbf{z}')$ and $u_3 = (c - c') \neq 0$. Since in valid signatures, $\mathbf{h}$ and $\mathbf{h}'$ have at most ω non-zero entries, according to Lemma 1 in [6], the vectors $A\mathbf{z} - c\mathbf{t}_1 \cdot 2^d - \mathbf{w}_1 \cdot 2\gamma_2$ and $A\mathbf{z}' - c'\mathbf{t}_1 \cdot 2^d - \mathbf{w}'_1 \cdot 2\gamma_2$ have at most ω coefficients with absolute values greater than γ_2 . Therefore, the vectors $\mathbf{u}_1, \mathbf{u}_2, u_3$ have at most 2ω coefficients with absolute values greater than $2\gamma_2$, i.e., $\mathbf{u}_2$ has at most 2ω coefficients with absolute values greater than $2\gamma_2$. In summary, extractor $\mathcal{E}$ has solved the MSIS problem in the lemma. $\square$

5.5 Theoretical security in the QROM

Theorem 3 (SUF-CMA security in the QROM). *In the Random Oracle Model (ROM), let $H(\rho) = \text{SampleInBall}(H_{2\lambda}(\mu \parallel \text{W1Encode}(\mathbf{w}_1)))$, where $H : \{0, 1\}^* \rightarrow B_\tau$ and the pseudorandom samplings are modeled as quantum random oracles. OPS-SIG is SUF-CMA secure under the MLWE and MSIS assumptions. Specifically, for any probabilistic polynomial-time adversary A , there exist probabilistic polynomial-time adversaries $\mathcal{B}, \mathcal{C}, \mathcal{D}$ such that:*

$$\text{Adv}_{\text{OPS-SIG}}^{\text{SUF-CMA}}(\mathcal{A}) \leq \text{Adv}_{n,q,k,l,\eta}^{\text{MLWE}}(\mathcal{B}) + \text{Adv}_{H,n,q,k,l+1,\zeta}^{\text{SelfTargetMSIS}}(\mathcal{C}) + \text{Adv}_{n,q,k,l,\zeta'}^{\text{MSIS}}(\mathcal{D})2^{-254}$$

where:

$$\begin{aligned}\zeta &= \max\{\gamma_1 - \beta_1, 2\gamma_2 + 1 + 2^{d-1} \cdot \tau\} \\ \zeta' &= \max\{2(\gamma_1 - \beta_1), 4\gamma_2 + 2\}\end{aligned}$$

Proof. The provable security in the ROM model above is non-tight because it uses the forking lemma. At the same time, because the forking lemma is used, this classical reduction proof cannot be directly extended to the quantum random oracle model (QROM). Despite this, so far, no signature scheme with a security proof obtained by using the forking lemma has its security reduced because the reduction process is non-tight. The reason is: there exists an intermediate state problem in the proof process, and even under quantum reduction, the reduction proof from the EUF-CMA security of the signature scheme to this intermediate state problem is tight. To some extent, this intermediate state problem is a mixed problem combining the underlying mathematical problem and the hash function. So far, all studies indicate that as long as the algebraic structure of the underlying mathematical problem and the hash function have no direct relationship, solving this intermediate state problem will not be easier than solving the underlying mathematical problem. For OPS-SIG, the corresponding intermediate state problem is the SelfTargetMSIS problem. Intuitively, the MLWE hardness assumption is used to prevent key recovery attacks, the MSIS hardness assumption is used to achieve strong unforgeability, and the SelfTargetMSIS hardness assumption is used to prevent signature forgery for new messages.

The proof of Theorem 3 can be obtained by modifying the proof in [6]. Next, we briefly explain the part related to the SelfTargetMSIS hardness assumption (the part related to the MSIS problem is similar to the proof in Lemma 2).

[6] first proved that in the quantum random oracle model, for digital signature schemes with zero-knowledge properties, EUF-CMA security can be reduced to EUF-NMA security, where EUF-NMA security does not allow the adversary to make signature queries (i.e., the adversary only obtains the public key). Under the MLWE hard assumption, OPS-SIG can be proved to have zero-knowledge properties. Therefore, we mainly analyze the second stage below, proving that when the underlying hard problem assumption holds, no effective attacker can forge a valid signature with non-negligible probability when only obtaining a random public key $(A, \mathbf{t})$ (in the quantum random oracle model, directly setting A as the public key is the same as setting a random seed ρ as the public key from a security perspective).

If adversary $\mathcal{A}$ can output a valid message-signature pair $(M, \mathbf{z}, \mathbf{h}, c)$ based on a known randomly sampled $(A, \mathbf{t})$, this signature must satisfy: $\|\mathbf{z}\|_\infty < \gamma_1 - \beta_1$, $c := H(\mu \parallel \text{UseHint}(\mathbf{h}, A\mathbf{z} -$

$c\mathbf{t}_1 \cdot 2^d, 2\gamma_2)$) The number of coefficients 1 in $\mathbf{h}$ is less than ω . According to the second item of Lemma 1 in [6], the following equation holds:

$$2\gamma_2 \cdot \text{UseHint}(\mathbf{h}, A\mathbf{z} - c\mathbf{t}_1 \cdot 2^d, 2\gamma_2) = A\mathbf{z} - c\mathbf{t}_1 \cdot 2^d + \mathbf{u}$$

where $\|\mathbf{u}\|_\infty \leq 2\gamma_2 + 1$, and $\mathbf{u}$ has only ω coefficients with absolute values greater than γ_2 ; the above equation can be written as:

$$A\mathbf{z} - c\mathbf{t}_1 \cdot 2^d + \mathbf{u} = A\mathbf{z} - c\mathbf{t} + c\mathbf{t}_0 + \mathbf{u} = A\mathbf{z} - c\mathbf{t} + \mathbf{u}'$$

where $\|\mathbf{u}'\|_\infty \leq \|c\mathbf{t}_0\|_\infty + \|\mathbf{u}\|_\infty \leq \tau \cdot \|\mathbf{t}_0\|_\infty + \|\mathbf{u}\|_\infty \leq 2^{d-1} \cdot \tau + 2\gamma_2 + 1$; therefore, the adversary outputting a valid message-signature pair $(M, (\mathbf{z}, \mathbf{h}, c))$ is equivalent to outputting a set $(M, (\mathbf{z}, c, \mathbf{u}'))$ satisfying the following conditions: $\|\mathbf{z}\|_\infty < \gamma_1 - \beta_1$, $\|c\|_\infty = 1$, $\|\mathbf{u}'\|_\infty \leq 2^{d-1} \cdot \tau + 2\gamma_2 + 1$, $H(\mu \parallel \frac{1}{2\gamma_2} \cdot [A] - \mathbf{t}[I_k] \cdot [\mathbf{z}|c|\mathbf{u}']^T) = c$. Define $H(\mu \parallel x) = H'(\mu \parallel 2\gamma_2 x)$, then the last condition is equivalent to:

$$H'(\mu \parallel [A] - \mathbf{t}[I_k] \cdot [\mathbf{z}|c|\mathbf{u}']^T) = c$$

Since $(A, \mathbf{t})$ is randomly sampled, the above problem is equivalent to the SelfTargetAMISIS problem. □

5.6 Concrete security

We evaluate the concrete security of OPS-SIG by considering the best-known families of attacks against the underlying lattice problems, MLWE and MSIS. The attacks include lattice-reduction-based approaches and algebraic/combinatorial approaches.

Primal Attack. The primal attack attempts to solve the MLWE problem by reformulating it as a unique Shortest Vector Problem (uSVP) in an embedded lattice. Given an MLWE instance $(\mathbf{A}, \mathbf{t} = \mathbf{A}\mathbf{s}_1 + \mathbf{s}_2)$, the adversary constructs a lattice spanned by the columns of a block matrix containing $\mathbf{A}$ and the identity matrix, augmented with the target vector $\mathbf{t}$. The goal is to find the unique short vector corresponding to $(\mathbf{s}_1, \mathbf{s}_2, 1)$ using block-Korkine-Zolotarev (BKZ) lattice reduction algorithms. The cost is determined by the block size b required for the BKZ algorithm to recover the short vector, which is typically estimated using the Core-SVP methodology. Both classical and quantum sieving models are considered for the underlying SVP subroutine within BKZ.

Dual Attack. The dual attack aims to solve the decisional MLWE problem by finding short vectors in the dual lattice of the module lattice generated by $\mathbf{A}$. The adversary searches for a short vector $(\mathbf{v}, \mathbf{w})$ such that $\mathbf{v}^T \mathbf{A} \equiv \mathbf{w}^T \pmod{q}$. Once found, the adversary computes the inner product $\mathbf{v}^T \mathbf{t} = \mathbf{v}^T (\mathbf{A}\mathbf{s}_1 + \mathbf{s}_2) = \mathbf{w}^T \mathbf{s}_1 + \mathbf{v}^T \mathbf{s}_2 \pmod{q}$. Since $\mathbf{s}_1, \mathbf{s}_2, \mathbf{v}$, and $\mathbf{w}$ are all short, this inner product is significantly shorter than a uniformly random element in $\mathbb{Z}_q$, thus yielding a distinguisher. The complexity of finding such short dual vectors is also estimated using BKZ reduction and the Core-SVP model.

SIS Attack. For the MSIS problem, the adversary seeks to find a short vector $\mathbf{z}$ such that $\mathbf{A}\mathbf{z} \equiv \mathbf{0} \pmod{q}$. This is directly formulated as a Shortest Vector Problem (SVP) in the q -ary lattice $\Lambda^\perp(\mathbf{A})$. The adversary runs the BKZ algorithm to find a vector shorter than the norm bound required by the signature scheme's forgery conditions. The cost is again modeled using the required BKZ block size b to achieve the target root Hermite factor.

Hybrid Attack. The hybrid attack combines lattice reduction and MITM techniques. It splits the secret vector into two parts: one part is guessed using a MITM search, while the remaining part is recovered by applying lattice reduction on a reduced-dimension lattice. This attack is particularly relevant when the secrets are very sparse or extremely short. For the parameter sets of OPS-SIG, the search space for the MITM phase is too large to offer a significant advantage over pure lattice reduction attacks.

BKW Attack. The Blum-Kalai-Wasserman (BKW) attack is a combinatorial algorithm that attempts to solve LWE by finding linear combinations of the rows of $\mathbf{A}$ that sum to zero (or very small vectors), thereby eliminating portions of the secret $\mathbf{s}_1$. While asymptotically subexponential, the BKW algorithm requires an exponential number of samples (which is limited in the standard LWE setting unless one uses the given fixed number of samples and applies sample amplification) and immense memory. For the modulus $q \approx 2^{26}$ and the dimensions used in OPS-SIG, BKW is highly inefficient and far outperformed by lattice-reduction attacks.

S-unit Attack. The S-unit attack targets ideal lattices and some structured module lattices (like NTRU) by exploiting the algebraic structure of the underlying number field. It relies on finding short elements in the S-unit group of the field to recover the secret key. However, OPS-SIG is based on the MLWE and MSIS problems with module ranks $k, \ell \geq 3$, which are known to resist such algebraic attacks. The S-unit attack does not currently apply to module lattices with rank strictly greater than 1 without significant degradation in performance.

Arora-Ge Attack. The Arora-Ge attack is an algebraic method that solves LWE by formulating it as a system of non-linear polynomial equations over $\mathbb{Z}_q$. Since the error components $\mathbf{s}_2$ are bounded by η_2 , they satisfy the polynomial equation $\prod_{e=-\eta_2}^{\eta_2} (x - e) = 0 \pmod{q}$. This attack is highly effective when the noise bound η_2 is extremely small relative to the number of available samples. However, the complexity grows exponentially with η_2 . For OPS-SIG, the noise bounds $\eta_1 = \eta_2 \in \{2, 3\}$ and the number of equations make the algebraic system far too complex to be solved faster than lattice reduction.

Security estimates. Table 2 summarizes the resulting bit-security estimates for each instance. Concrete security estimates are derived by reducing MLWE/MSIS instances to approximate shortest vector problems via primal/dual lattice attacks (BKZ reduction with sieving). We report security in bits as the base-2 logarithm of the best-known attack cost.

Table 2: Concrete security estimates (in bits) for underlying lattice problems.

Instance	weak SIS (classical/quantum)	strong SIS (classical/quantum)	LWE (classical/quantum)
Level-1 (128/80)	167 / 146	152 / 133	178 / 156
Level-3 (256/128)	278 / 244	257 / 225	269 / 236
Level-5 (512/256)	567 / 497	520 / 455	595 / 521

6 Performance Evaluation

6.1 Asymptotic performance

The dominant operations are the matrix-vector multiplications in $\mathcal{R}_q$, implemented efficiently using NTT. Key generation performs one $k \times \ell$ matrix-vector multiplication. Verification performs one $k \times \ell$ matrix-vector multiplication and one multiplication by the challenge polynomial. Signing performs one matrix-vector multiplication per rejection iteration; the expected number of iterations is given in Table 1.

6.2 Software benchmark results (reference implementation)

This section reports measurements for the reference implementation under `Implementations/Reference_Implementation/AlgorithmInstance`, which corresponds to Level-1 parameters in Table 1. The benchmark harness is `test_speed_yuandunsig.c`, compiled with `gcc -O2 -std=c99`.

Platform configuration.

- CPU: Intel(R) Xeon(R) Platinum 8575C (4 logical CPUs, 2 cores, SMT enabled)
- OS: Linux 5.15.0-71-generic (x86_64)
- Compiler: gcc 11.4.0 (Ubuntu 22.04)

Timing results. Table 3 lists the minimum, median, and average timings in nanoseconds.

Table 3: Reference implementation timings for the implemented instance (Level-1).

Operation	min (ns)	med (ns)	avg (ns)
<code>polyvec_matrix_expand</code>	526978	625492	709577.5
<code>poly_uniform_eta</code>	26726	34668	35623.5
<code>poly_uniform_gamma1</code>	61788	66818	67237.1
<code>poly_ntt</code>	33110	36842	38403.6
<code>poly_invntt_tomont</code>	41311	46064	46600.4
<code>poly_pointwise_montgomery</code>	835	1252	8973.3
<code>poly_challenge</code>	15373	32394	30062.3
KeyGen	973089	1003414	1133165.3
Sign	2123673	2389713	2643769.6
Verify	983209	1483540	1511314.6

7 Feature Statement

7.1 Declared features

- **Innovation.** The scheme reduces public-key and signature sizes while maintaining strong security margins [2]. The reference-oriented specification additionally highlights efficient sparse-challenge sampling (`SampleInBall`) tailored to $n \in \{512, 1024\}$ and small τ .
- **Simplicity.** The design follows the well-studied ML-DSA structure [1], separating sampling, rounding/decomposition, hinting, and packing into explicit auxiliary algorithms, which supports independent review and faithful implementations.
- **Flexibility.** The specification supports multiple parameter sets and keeps parameter-dependent parts explicit via $(k, \ell, n, \eta, \gamma_1, \gamma_2, \tau, \omega)$.
- **Compatibility.** The API is a standard signature interface (KeyGen/Sign/Verify). Optional context binding is supported and is hashed into the signed transcript to mitigate cross-protocol replay.
- **Extensibility.** The underlying MLWE/MSIS setting enables a coherent suite approach. With consistent parameter methodology one can build matched-strength KEM and key-exchange variants over the same ring and modulus family.

- **Performance across platforms.** The scheme is designed for software and hardware efficiency: NTT-based arithmetic enables fast polynomial multiplication; sparse challenges enable shift-add multiplications and early-abort checks; these operations admit parallelization and benefit from vector/SIMD instructions and hardware accelerators.

7.2 Security declarations

- **Theoretical security.** The scheme targets SUF-CMA security in the (Q)ROM under MLWE and MSIS, following Fiat-Shamir with aborts [1].
- **Concrete security.** The three instances are intended to meet classical security targets of 128/256/512 bits with corresponding quantum targets of at least 80/128/256 bits, with additional conservative margin reflected in the concrete estimates.

References

- [1] National Institute of Standards and Technology. Fips 204: Module-lattice-based digital signature standard. National Institute of Standards and Technology, 2024.
- [2] Jiang Zhang, Yu Yu, Shize Fan, et al. Tweaking the asymmetry of asymmetric-key cryptography on lattices: KEMs and signatures of smaller sizes. In *Public-Key Cryptography – PKC 2020*, pages 37–65. Springer International Publishing, 2020.
- [3] Standardization Administration of China. GB/T 32905-2016: Information security technology – SM3 cryptographic hash algorithm. Standardization Administration of China, 2016.
- [4] Standardization Administration of China. GB/T 15852.2-2024: Information technology – security techniques – key derivation functions. Standardization Administration of China, 2024.
- [5] Jieyu Zheng, Feng He, Shiyu Shen, Chenxi Xue, and Yunlei Zhao. Parallel small polynomial multiplication for dilithium: A faster design and implementation. In *Annual Computer Security Applications Conference, ACSAC 2022, Austin, TX, USA, December 5-9, 2022*, pages 304–317. ACM, 2022.
- [6] Shi Bai, Léo Ducas, Eike Kiltz, Tancrede Lepoint, Vadim Lyubashevsky, Peter Schwabe, Gregor Seiler, and Damien Stehlé. Crystals-dilithium algorithm specifications and supporting documentation(version 3.1). 2021.